2_text_prep

January 19, 2026

1 Pre-Lab Instructions

This poor man will be at the start of every notebook, letting you know what you will need for the lab.

For this lab you will need: - DATA: `farright_dataset.parquet` - Download from Moodle and upload to this Colab session. - IF YOU'RE *NOT* USING COLAB - You will need to install `spacy` and `beautifulsoup4` and the spacy model, use the cell below.

[ ]:
```
#*
# If you are NOT using Google Colab you'll need to uncomment the lines below␣
 ↪and run this cell to install spacy and its model
# import sys
# ! pip install spacy beautifulsoup4
# !{sys.executable} -m spacy download en_core_web_sm
```

2 SC290: How to Clean and Care for your Text

Welcome back to coding for social science research! You're here again so presumably you enjoyed your introduction in SC207, and you're ready to apply those skills to data analysis techniques that will make you look like a wizard.

In today's lab we will cover:

Cleaning text

Real world text is messy, filled with symbols and stuff you don't actually want. We'll look at how to isolate the text you want and clean away the stuff you don't.

Tokenising text

Most text analysis techniques rely on breaking up long strings into small chunks, such as into individual words or 'tokens'. We'll look at one way of tokenising text to prep it ready for analysis.

3 1. Cleaning Text

Text cleaning is the process of removing parts of a text to only leave the 'content' that matters to you for your particular use-case. Text collected from APIs, scraped from websites, or found in existing datasets is often messy, can contain weird symbols/characters, may include special symbols that help with the formatting and display of the text when it is on a website etc.

Cleaning texts is very source specific, meaning exactly what you need to do to text can vary a lot depending on where it came from, and your particular needs.

Today in cleaning we're going to focus on three things. - Using the HTML formatting in a piece of text to isolate and remove parts that are not part of the primary content. - Replacing specific characters that are unusual and will be misunderstood in later text processing packages. - Cleaning out the HTML formatting to leave us with just plain text.

> **About the Dataset**
>
> `farright_dataset.parquet` is a dataset of articles from The Guardian API, retrieved and prepped using the processes we used in SC207. - Retrieving from the API using the simple query of `"far-right"` with a limit of 1,500 articles, ordered newest first. - Only 'articles' from the 'News' pillar were retained. - Unpacking nested data into its own columns and setting the correct data types - Removing articles that were outliers such as sponsored content

[ ]:
```
# Let's import our libraries and load the dataset

import pandas as pd
from bs4 import BeautifulSoup

articles = pd.read_parquet('farright_dataset.parquet')
articles.info()
```

[ ]:
```
# We turn our pandas column of texts into a simpler list to make it compatible␣
 ↪with BeautifulSoup and Spacy
texts = articles['body'].tolist()
```

3.1 1.1 Isolating and removing irrelevant parts

Run the cell below and take a look at an example of the text of the article sent back by the API.

[ ]:
```
#*
# For teaching purposes only - finds article with an <aside> element in
idx = articles[articles['body'].str.contains('<aside')].last_valid_index()
test_text = texts[idx]


# Prints out the URL of the story so we can view it as it's meant to look and␣
 ↪compare to the text we have.
print(articles.loc[idx,'webUrl'])
print('----')
print(test_text)
```

3.1.1 About HTML

This story is formatted using HTML, the language used to represent website content.

Websites are made up of 'elements' which are defined by wrapping pieces of text between tags to

show where the element begins `< >` and ends `</ >`. Whilst HTML has become more complex since its original design, generally content is wrapped in an element to control how it is displayed.

For example `<p>These tags indicate content is a paragraph</p>`.

Sometimes an element will also have a `class` which tells the website to format that content differently.

```
<p class="important_highlight_big"> The most important point of our story is.... </p>
```

Sometimes an element can be *inside* another element

```
<p>Paragraph content is here talking about an important <aside>See our great new dog story</asi
```

3.1.2 Using HTML to navigate text

We simply want the text inside the most basic 'paragraph' `<p>` elements, but we also *don't* want any side content that might be embedded inside a paragraph element.

To do this we will `decompose` certain elements, i.e. isolate them and cut them out of the text, before then identifying all the `<p>` elements and getting their text.

The library `BeautifulSoup` is designed to read HTML text and turn it into a structured object that we can navigate, and manipulate.

[ ]:
```
# First we take the text and use Beautiful soup to interpret it
# We call the result 'soup', because that's what people do.
soup = BeautifulSoup(test_text, 'html.parser')
```

[ ]:
```
# We can ask for all the 'p' paragraph elements.
# This also will show us what other elements are embedded inside them.
soup.find_all('p')
```

- There are `span` and `aside` elements. We want to remove them and the text inside them.
- There are also `<a>` elements which are how HTML indicates something is a link. The text wrapped by an `<a>` element makes that text a link, but it is also still a part of the main text, so we want to leave those in place.

[ ]:
```
# First we will remove unwanted elements
# We'll make a list of element types we want to remove
unwanted_elements = ['span','aside']

# Next we use .find_all() to iterate over every unwanted element,
# and .decompose() it - delete it.
for element in soup.find_all(unwanted_elements):
    element.decompose()
```

3.1.3 List Comprehensions

In this module we're going to start using something called *List Comprehensions*. These are an alternative way of looping over a list to filter or edit its contents. Rather than the usual:

```
old_list = ['a','b','c']
new_list = []

for item in old_list:
    new_item = do_thing(item)
    new_list.append(new_item)
```

We can instead keep it cleaner in one line:

```
new_list = [do_thing(item) for item in old_list]
```

The general structure of a list comprehension is:

```
[item_to_keep for item in list_to_iterate_over]
```

The job we did above could also be a list comprehension, we just don't assign the result to a variable. `.decompose()` doesn't produce any value, it just deletes an element so the list would be empty anyway.

[ ]:
```
# Same decompose job as above but in one line
[element.decompose() for element in soup.find_all(unwanted_elements)]
```

Now we've decomposed the elements we don't want, we just need to extract the text from every `p` element

[ ]:
```
# and we'll then retain the text associated with any p element that has no␣
 ↪associated class
paras = [p.text for p in soup.find_all('p')]
paras
```

[ ]:
```
# finally we convert that list of strings into a single string,
# retaining the paragraph breaks by inserting a new line break between each␣
 ↪paragraph.
cleaned_item ='\n'.join(paras)
print(cleaned_item)
```

3.2 1.2 A text cleaning function

We can do this for every article in our list. First we'll build a function to do the job of cleaning, then we'll apply it to every item in the list of texts. But first…

[ ]:
```
#*
# The Guardian sometimes uses this character rather than a normal apostrophe.
# Tiny things like this can really throw off text analysis, so we'll fix it in␣
 ↪our function.
annoying_character = "’"
```

[ ]:
```
def clean_guardian_text(text, remove_elements=['span','aside']):
    soup = BeautifulSoup(text, 'html.parser')
    [e.decompose() for e in soup.find_all(remove_elements)]
    paras = [p.text for p in soup.find_all('p')]
```

```
    cleaned_item ='\n'.join(paras)
    cleaned_item.replace("’", "'") # replacing an annoying character used in␣
 ↪the guardian
    return cleaned_item

cleaned_texts = [clean_guardian_text(t) for t in texts]
```

[ ]:
```
print(cleaned_texts[0])
```

3.3 1.3 Saving your cleaned text

[ ]:
```
#*
articles['cleaned_text'] = cleaned_texts
articles.to_parquet('farright_dataset_cleaned.parquet')
```

4 2. Tokenising

A lot of text analysis relies on making texts into 'tokens'. A simple way of thinking about this is that it splits a text into individual words.

`"Hello, how are you?"` might become `['Hello','how','are','you']`.

This is a more complicated task than you might think, because it could also become: - `['Hello,','how','are','you?']` - `['Hello', ',','how','are','you', '?']`

To help us we need a library dedicated to handling text - enter spaCy

[ ]:
```
#*
import spacy
nlp = spacy.load('en_core_web_sm')
doc = nlp(cleaned_texts[-1])
doc
```

Whilst this may look the same, it is now a spacy `Doc` object that understands the text and can help us break it down into tokens.

[ ]:
```
#*
# This is how spacy breaks up the document
[t for t in doc][:10]
```

[ ]:
```
#*
# # Spacy uses the context of the surrounding words and grammar to work out if␣
 ↪the word is a noun, verb, adjective etc.
# They call this the 'part-of-speech' or POS
[(t, t.pos_) for t in doc][:10]
```

[ ]:
```
#*
# Spacy tokens have helpful attributes...
# Is it alphabetical (i.e not numerical or punctuation)
```

```
[(t.text, t.is_alpha) for t in doc][:10]
```

[ ]:
```
#*
# Is it punctuation?
[(t, t.is_punct) for t in doc][:15]
```

[ ]:
```
#*
# # Is it a stop word?
[(t, t.is_stop) for t in doc][:10]
```

4.1 2.1 Stop Words?

Stop words are typically defined as the most common words in a language. Often incredibly common words can make it harder to find patterns in text. For example the most common words in a piece of text might be 'the', 'a', 'and' etc. That doesn't tell us much about the text even though the result is correct.

[ ]:
```
#*
# These are the stop words for this model
print(nlp.Defaults.stop_words)
```

[ ]:
```
# We can use these token attributes to filter our text based on what type of␣
 ↪token it is

# This ensures only alphabetical tokens that aren't stop words are retained.
[t for t in doc if t.is_alpha and not t.is_stop]
```

[ ]:
```
# This allows numbers as well, but filters out space symbols like \r and \n and␣
 ↪punctuation

[t for t in doc if not t.is_space and not t.is_punct and not t.is_stop]
```

4.2 2.2 Lemmatization

A word's lemma is the simpler 'root' word that best represents the word's meaning. It reduces the possible range of words whilst still ensuring the words left convey the appropriate meaning.

To make this clearer we can use some examples:

[ ]:
```
#*
# Here we have essentially the same sentences, just a variation in that one␣
 ↪uses a contraction "don't" rather than "do not".
rabbit_1 = nlp("I don't like rabbits in space")
rabbit_2 = nlp("I do not like rabbits in space")
print( [token.lemma_ for token in rabbit_1])
print( [token.lemma_ for token in rabbit_2])
```

```
[ ]: #*
     # Even differing text can be brought at least closer in similarity using␣
      ↪lemmas, reducing loving to love
     rabbit_1 = nlp("I'm loving these rabbits")
     rabbit_2 = nlp("I love this rabbit!")

     print( [token.lemma_ for token in rabbit_1])
     print( [token.lemma_ for token in rabbit_2])
```

If you are doing any text analysis that counts the frequency of words, relies on word similarity etc, it is usually a good idea to reduce the range of words being used so long as it can retain the same underlying semantic meaning.

```
[ ]: filtered_tokens = [t.lemma_.lower() for t in doc if not t.is_stop and t.
      ↪is_alpha]
     filtered_tokens
```

```
[ ]: #*
     # We can very quickly get a sense of a document's content by looking at the␣
      ↪most common tokens
     from collections import Counter
     counts = Counter(filtered_tokens)
     counts.most_common(10)
```

```
[ ]: # If you want to convert your filtered tokens to text you simply join them␣
      ↪together again
     filtered_text = " ".join(filtered_tokens)
     filtered_text
```

4.3 2.3 Tokenising in bulk

Spacy does some pretty heavy lifting so we should tokenise once, and then save the result to avoid having to rerun the process again. Spacy also has a method that speeds up tokenising on large numbers of documents called `.pipe`.

`.pipe` efficiently processes large amounts of documents by: - Handling them in large batches controlled by the `batch_size=` argument. - Running multiple workers at the same time, controlled by the `n_process=` argument.

Now we're getting into analysis we're going to start encountering instances where the efficiency of code matters, because the jobs we're doing are increasingly more complex and take longer to complete.

4.3.1 What settings do I use?

Colab Colab takes around 4 minutes to process 500 articles. To avoid issues you should set: - `batch_size=150` - `n_process=1`

Your own computer Entirely down to the hardware you use. If you have a more powerful laptop with multiple CPU cores you can increase the number of workers to 1 less than the number of cores you have. If you have a lot of RAM you can increase the batch size.

[ ]:
```
#*
# Let's do a quick reset here just to clean up any issues you may have had
import pandas as pd
import spacy

articles = pd.read_parquet('farright_dataset_cleaned.parquet')
cleaned_texts = articles['cleaned_text'].tolist()
nlp = spacy.load('en_core_web_sm')
```

[ ]:
```
#*
# We'll use this function as our main tokeniser
# It takes in a single spacy Doc and outputs a single string of tokens
def tokenise_doc(doc: spacy.tokens.Doc) -> str:
    tokens = [t.lemma_.lower() for t in doc if not t.is_stop and t.is_alpha]
    return ' '.join(tokens)

example = nlp(cleaned_texts[-1])
tokenise_doc(example)
```

[ ]:
```
# Now we will create a 'pipe' that will efficiently transform a list of texts
# into spacy Doc objects.
# We will process each Doc using our function, and store the results.

BATCH_SIZE = 150
WORKERS = 1

tokenised_documents = []

for doc in nlp.pipe(cleaned_texts, batch_size=BATCH_SIZE, n_process=WORKERS):
    tokenised_documents.append(tokenise_doc(doc))

articles['tokens'] = tokenised_documents
```

[ ]:
```
#*
# We can see the result and compare with the cleaned text

example_row = articles.iloc[-1]
print(example_row['cleaned_text'][:250])
print('****')
print(example_row['tokens'][:250])
```

4.4 2.4 Saving

[ ]:
```
#*
# We save our new version of our dataframe with the tokens column.
# We'll be using these in the next few sessions.
articles.to_parquet('farright_dataset_cleaned.parquet')
```

5 Summary

Today we covered: - HTML code and how to use it to isolate parts of web based text. - How to extract the text from HTML code. - Tokenising, why it is tricky to do. - Stop words and word lemmas - Bulk tokenisation of large text datasets ready for analysis.

Next week we will look at how tokens can be used to represent groups of documents and help us determine document similarity and difference. These are the foundations of text analysis.